MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Institution of Manipal University

MANIPAL INSTITUTE OF TECHNOLOGY

(A Constituent Unit of Manipal Academy of Higher Education)

School of Computer Engineering

Foundations of Computer Vision Lab (CSE-3176)
Mini-Project Report

Project Title

Vision-Assisted Survivor Localisation in Maritime Search and Rescue using Texture-Anomaly Modelling

Submitted by

Name	Roll No.	Registration No.
Tanish Hemant Kuvar	47	240962540
M Vishal Kumar	49	240962546
Samyak Jain		240962638

Academic Year 2026–2027

Vision-Assisted Survivor Localisation in Maritime Search and Rescue using Texture-Anomaly Modelling

Tanish Hemant Kuvar
Reg. No. 240962540
AIML, Section A

M. Vishal Kumar
Reg. No. 240962546
AIML, Section A

Samyak Jain
Reg. No. 240962638
AIML, Section A

***Abstract*—People in the water seen from a drone often cover fewer than 16 pixels, too few for appearance-based recognition. This paper presents a search-and-rescue (SAR) aid that models the water rather than the person. Each aerial RGB frame passes through six classical stages: illumination normalisation, Gabor and uniform local binary pattern (LBP) texture description, chi-square anomaly scoring against a robust local reference, glint suppression by chromaticity, morphological shape filtering, and temporal tracking. The output is a ranked shortlist of at most k regions per frame, shown to an operator through a Tkinter desktop GUI and a browser-based console. No training data is required. On a synthetic benchmark of 6 sequences (360 frames, 739 annotated people), the system reaches 92.4% recall@5 with 51.7% precision and 1.77 false alarms per frame, rising to 99.3% recall once the tracker has warmed up, at about 2.7 frames/s on a laptop CPU. An ablation shows that the temporal stage cuts false alarms by 35%. These results validate the pipeline end to end; real-world performance remains to be measured on public datasets.**

***Index Terms*—search and rescue, anomaly detection, Gabor filters, local binary patterns, morphological filtering, multi-frame tracking**

I. Introduction

When a person falls overboard, a drone can cover a large area of sea quickly, but an operator watching the feed must find a head-sized target among waves, sun glint and foam. Learned detectors need large annotated datasets and can fail when the sea state, altitude or camera differs from training. This work instead treats survivor search as anomaly detection: it estimates what normal sea looks like in the current frame and reports regions that do not fit.

The contributions are:

1) a six-stage pipeline built only from classical image processing operations, with no training step;
2) a texture reference estimated per frame from a local median, which stays clean when targets are present;
3) colour, shape and temporal tests that remove glint, foam and whitecaps;
4) two operator front ends that expose every intermediate stage and parameter;
5) an evaluation with recall@k, false alarms per frame and stage-by-stage ablation.

TABLE I
System Requirements

Type	Requirement	How it is met
Functional	Accept video, an image folder or an image	`run` command and GUI loaders
Functional	Output a ranked top-k list as overlay and CSV	`run --output --csv`; shortlist panels
Functional	Inspect stages and tune parameters	Stage selector, 16 sliders, JSON configs
Functional	Provide a GUI front end	Tkinter GUI and browser console
Functional	Evaluate on annotated data	`evaluate`; COCO and YOLO loaders
Non-funct.	Run on a CPU, no GPU	$\approx$2.7 frames/s at 1280×720
Non-funct.	Repeatable and tested	40 passing unit and integration tests
Software	Python, NumPy $\geq$1.24, OpenCV $\geq$4.8, Pillow, Matplotlib, pytest, Tkinter	`requirements.txt`
Input	Daylight aerial RGB of open water	Night and thermal out of scope

II. Problem Statement

Given a sequence of daylight aerial RGB frames of open water, return for each frame a ranked list of at most k regions likely to contain a person, each with a confidence value. Three properties make this hard:

- *Small targets:* 618 of the 739 annotated people in the benchmark measure under 16 px (square root of box area).
- *Moving clutter:* waves, sun glint, whitecaps and debris all differ from calm water and change from frame to frame.
- *No fixed appearance:* a partly submerged person may show only a head, and clothing colour varies.

A. System Requirements

Table I lists the requirements and how the system meets them.

III. Methodology

Stages 1 to 5 run on each frame independently; stage 6 uses the history across frames. Fig. 1 shows the data flow.

The filtering operations of stages 1, 2, 4 and 5 are detailed in Section IV. This section describes how they combine into a decision.

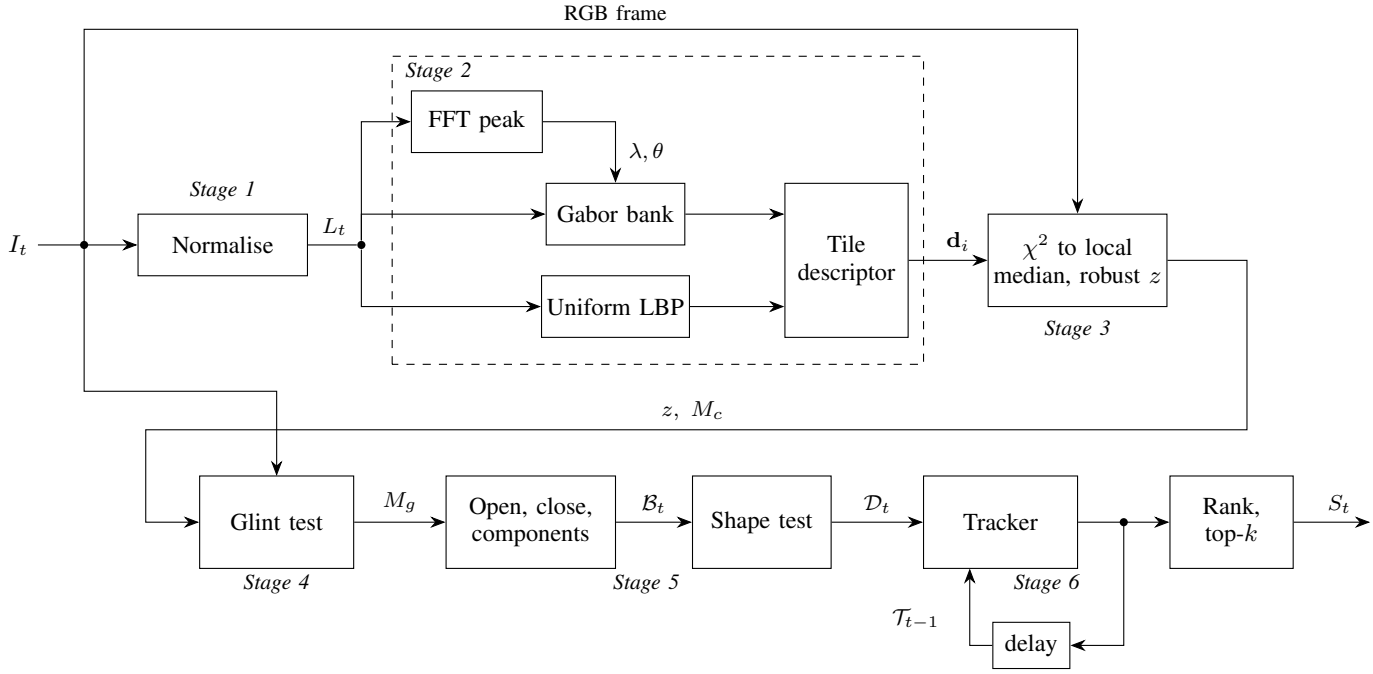


Fig. 1. Block diagram of the pipeline. I_t: input frame; L_t: normalised lightness; λ, θ: dominant wavelength and orientation; $\mathbf{d}_i$: descriptor of tile i; z: anomaly map; M_c, M_g: candidate mask before and after the glint test; $\mathcal{B}_t$: blobs; $\mathcal{D}_t$: detections; $\mathcal{T}_{t-1}$: previous tracks; S_t: shortlist.

A. Tile Descriptors

The normalised lightness is divided into overlapping square tiles of side T with stride $T/2$, so a person on a tile border is not split. With automatic sizing, T is about twice the expected person size: 32 px at 1920 px frame width, scaled with frame width and limited to 16–64 px. Each tile descriptor holds the mean Gabor energy of every filter and the normalised 59-bin uniform LBP histogram [7].

B. Anomaly Scoring (Stage 3)

The reference for normal sea is the median descriptor over 9×9-tile windows, interpolated back to every tile. A median stays clean when a person or a boat covers some tiles, and the local window adapts to near and far field. Each tile is compared with its reference by the chi-square distance, computed separately for LBP and for reference-normalised Gabor energy:

$$\chi^2(\mathbf{a}, \mathbf{b}) = \frac{1}{2} \sum_i \frac{(a_i - b_i)^2}{a_i + b_i}. \tag{1}$$

The two distances are put on one scale with a robust z-score and averaged with equal weight:

$$z = \frac{d - \mathrm{median}(d)}{1.4826 \cdot \mathrm{MAD}(d)}. \tag{2}$$

The median absolute deviation (MAD) is used because targets are outliers that would inflate a standard deviation. Each pixel takes the maximum z of the tiles covering it, and the anomalous region is $z > 3$. Inside it, candidate pixels are those whose Lab colour differs from a median-filtered local sea colour by more than $\mathrm{median} + 4\sigma_{\mathrm{MAD}}$, which turns a coarse tile into a pixel-accurate blob.

C. Temporal Association and Ranking (Stage 6)

Detections are linked between frames by greedy nearest neighbour around a constant-velocity prediction within a 40 px radius. A track is promoted after 5 hits if its drift is coherent:

$$c = \exp\left(-\frac{s}{6\,\mathrm{px}}\right) \geq 0.25, \tag{3}$$

where s is the RMS deviation of the step velocities from their mean. Gaps of up to 3 frames are tolerated, covering a head briefly under a wave. Optional camera-motion compensation estimates the frame-to-frame translation by phase correlation. Promoted tracks are ranked by

$$\mathrm{conf} = 0.4\,\mathrm{conf}_{\mathrm{anom}} + 0.6\,\frac{\min(h, 10)}{10}\,c, \tag{4}$$

where h is the number of hits and $\mathrm{conf}_{\mathrm{anom}}$ an exponential moving average of the blob confidence; the result is multiplied by 0.8 per missed frame. The top k tracks form the shortlist.

IV. Image Enhancement and Filtering Techniques

Every stage uses classical intensity, spatial, frequency or morphological operations [1]–[3], implemented with OpenCV [4]–[6] and listed here in pipeline order.

A. Illumination Normalisation (Stage 1)

Three steps run on the Lab lightness channel so that texture statistics are comparable across the frame:

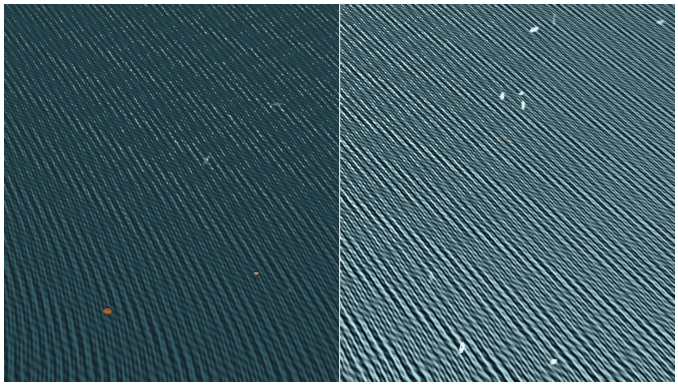


Fig. 2. A frame before (left) and after (right) normalisation.



Fig. 3. Even Gabor kernels at the four orientations of one wavelength.

1) *Flat-fielding:* lightness is divided by a heavily Gaussian-smoothed copy of itself (σ = 1/8 of frame width), removing the sun-side and far-field brightness gradient.
2) *CLAHE:* contrast-limited adaptive histogram equalisation with clip limit 2.0 over 8×8 regions.
3) *Piecewise-linear stretch:* a three-segment transform maps the 1st and 99th percentiles to 10 and 245, so a few glint pixels do not compress the rest.

Fig. 2 shows that the brightness gradient disappears while wave texture is kept.

B. Frequency-Domain Wave Estimate

The frame is split into far, middle and near bands. For each band, the 2-D DFT of a Hann-windowed patch gives a power spectrum, and its strongest peak with a wavelength of 4–32 px gives the dominant wavelength λ and orientation θ. This tunes the Gabor bank to the waves actually present.

C. Gabor Filtering

For each band wavelength the bank has four orientations, θ, $\theta+45°$, $\theta+90°$ and $\theta+135°$, with $\sigma = 0.56\lambda$ and aspect ratio $\gamma = 0.5$. Each filter is applied as an even and odd pair, and the local energy $\sqrt{e^2 + o^2}$ is used because it does not depend on wave phase. Long-wavelength filters run on a downsampled image at no loss in tile averages. Fig. 3 shows the even kernels of one wavelength.

D. Uniform Local Binary Patterns

Each pixel is compared with its eight neighbours (bit = 1 if neighbour $\geq$ centre), and the code is mapped to one of 59 uniform bins [7]. The per-tile histogram describes micro-texture independent of absolute brightness.

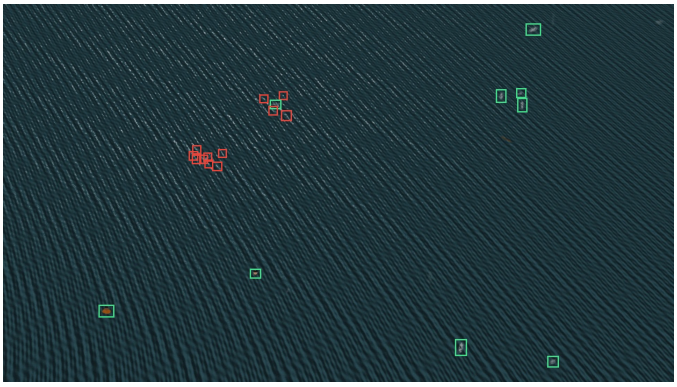


Fig. 4. Blobs after morphology: green passed the shape filter, red were rejected.

E. Median Filtering and Colour Distance

A median-filtered copy of the frame gives the local sea colour while ignoring small objects. Distances to it are measured in Lab space.

F. Glint Suppression by Chromaticity (Stage 4)

Specular glint saturates all channels together, so its rgb chromaticity $(R, G, B)/(R+G+B)$ lies close to $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. A pixel is glint if its mean is at least $0.8 \cdot 255$ and its chromaticity distance from grey is at most 0.06. Requiring both conditions keeps dark, colourless targets such as a black wetsuit.

G. Morphological Filtering (Stage 5)

Opening with a 3×3 ellipse removes specks and closing with a 5×5 ellipse joins the fragments of a partly submerged body. Connected components (8-connectivity) must have an area of 12–2500 px, eccentricity ≤ 0.97 from second-order moments (rejecting foam streaks) and solidity, the ratio of area to convex-hull area, ≥ 0.45 (rejecting ragged foam). Each blob is scored s = peak tile z $\times$ mean colour contrast, with confidence $s/(s+50)$. Fig. 4 shows kept and rejected blobs.

H. Mapping to Lab Sessions 1–5

Table II maps each technique to the function that implements it.

V. UI Design

Two front ends run the same pipeline and share one configuration. The Tkinter desktop GUI suits inspecting and tuning a single frame; the browser console analyses a whole sequence and plays it back, which suits a live demonstration.

A. Desktop GUI

The desktop GUI (Fig. 5) places the frame in the centre, input, playback, stage selection, parameters and export on the left, and the ranked shortlist with frame diagnostics on the right. The display-stage dropdown shows any of ten views, from the input to the final overlay. For the frame shown, the diagnostics report 321 of 6254 tiles anomalous and 9 of 26 blobs kept, with 3 tracks on the shortlist.

TABLE II
LAB TECHNIQUES AND THEIR IMPLEMENTATION

Technique	Implemented in
Intensity transform: piecewise-linear stretch	`normalise.piecewise_linear`
Histogram processing: CLAHE	`normalise.normalise`
Spatial filtering: Gaussian, median, Gabor convolution	`normalise.flat_field`, `anomaly.colour_deviation`, `texture.gabor_energy`
Frequency domain: 2-D DFT, phase correlation	`texture.estimate_wave`, `pipeline.estimate_shift`
Texture: uniform LBP, chi-square distance	`texture.uniform_lbp`, `anomaly.chi_square`
Colour spaces: Lab, rgb chromaticity	`anomaly.colour_deviation`, `glint.chromaticity`
Thresholding: median $+ k{\cdot}$MAD	`anomaly.score_anomalies`
Morphology: opening, closing	`morphology.refine`
Components, moments, convex hull	`morphology.refine`, `eccentricity`, `solidity`

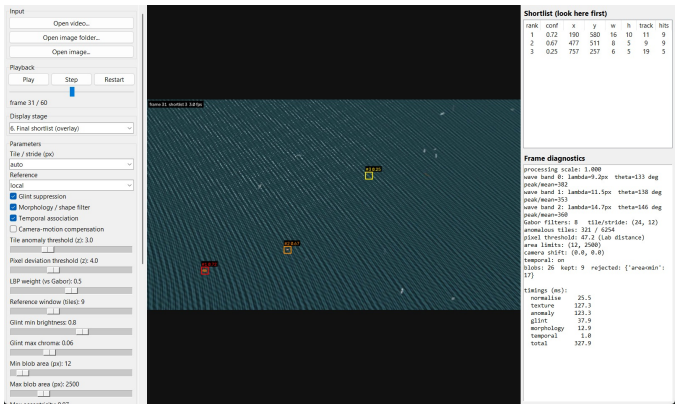


Fig. 5. Desktop GUI on sequence 5, frame 31. The shortlist table gives rank, confidence, box, track id and hits.

B. Web Operator Console

The console (Fig. 6) is started with `python -m sarvision web` and served at `localhost:8765` by a standard-library HTTP server. Stages 1–5 run in parallel on a thread pool while stage 6 runs in order, and the browser plays the result back at the source frame rate as the timeline fills in. The header shows sequence-level recall, precision, false alarms and throughput; the right panel shows ranked shortlist cards with crops, frame telemetry (wave-spectrum dial, per-stage timings, tile and blob counts) and tuning controls with ablation presets. A stage strip under the timeline opens any intermediate result for the current frame.

VI. VS CODE FOR FUNCTIONAL DESIGN

The system is a Python package, `sarvision`, of about 2900 lines developed in VS Code, with one module per stage and one shared configuration dataclass so the GUIs, command line and evaluation run identical code (Table III).

The application is run as follows:

```
pip install -r requirements.txt
```

TABLE III
MODULES OF THE `SARVISION` PACKAGE

File	Responsibility
`config.py`	All tunable parameters
`normalise.py`	Stage 1: flat-field, CLAHE, stretch
`texture.py`	Stage 2: DFT, Gabor bank, LBP, tiles
`anomaly.py`	Stage 3: reference, χ^2, robust z
`glint.py`	Stage 4: chromaticity glint test
`morphology.py`	Stage 5: morphology, shape filter
`tracking.py`	Stage 6: association, ranking
`pipeline.py`	Stage order; motion compensation
`gui.py`, `viz.py`	Tkinter GUI; stage rendering
`web/`	Browser console server and page
`cli.py`	Command-line entry point
`datasets.py`, `metrics.py`, `evaluate.py`	Loaders, metrics, ablations
`synthetic.py`	Synthetic benchmark generator

```
python -m sarvision synth --out data/synthetic
python -m sarvision gui data/synthetic/seq02.mp4
python -m sarvision web data/synthetic/seq05.mp4
python -m sarvision run --input data/synthetic/seq02.mp4 \
    --output out.mp4 --csv shortlist.csv
python -m sarvision evaluate --coco data/synthetic/
     annotations.json \
    --ablations full,no_temporal,no_glint,no_morphology
```

The core of stages 2 and 3 is shown below.

```
def gabor_energy(gray, filt):
    f = gray.astype(np.float32)
    e = cv2.filter2D(f, cv2.CV_32F, filt.even,
                     borderType=cv2.BORDER_REFLECT)
    o = cv2.filter2D(f, cv2.CV_32F, filt.odd,
                     borderType=cv2.BORDER_REFLECT)
    return cv2.magnitude(e, o)    # sqrt(e^2 + o^2)

def chi_square(a, b):
    return 0.5 * np.sum((a - b) ** 2 / (a + b + EPS),
                        axis=-1)

def robust_z(x):
    med = np.median(x)
    mad = np.median(np.abs(x - med)) * MAD_TO_SIGMA
    if mad < EPS:
        mad = x.std() + EPS
    return (x - med) / mad
```

VII. RESULTS AND DISCUSSION

A. Evaluation Protocol

The benchmark has 6 rendered sequences of 60 frames at 1280×720 and 10 fps with 739 annotated people, covering altitudes of 50–150 m, calm to rough sea, strong glitter, a dark-clothed person and a panning camera. A prediction is a hit when its centre lies within $\max(8\,\text{px}, \max(w, h))$ of the ground-truth centre, since IoU is too harsh for 10–30 px boxes. Steady-state figures exclude the first 4 frames of each sequence, before the tracker can promote any track. *All results are synthetic*: they validate the pipeline and evaluation, not real-world performance.

B. Ablation

Table IV compares the full system with one stage switched off at a time.

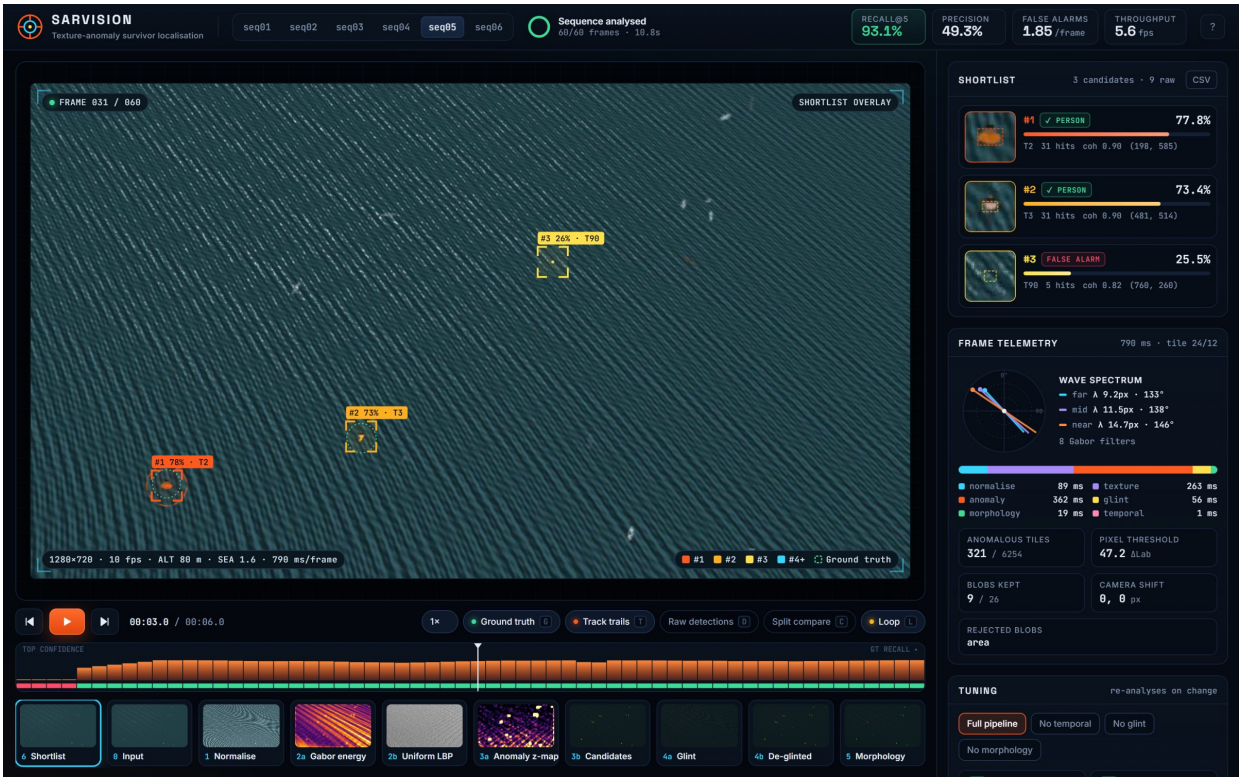


Fig. 6. Web console on sequence 5, frame 31. Two people are ranked #1 and #2 (77.8% and 73.4%); #3 is a short-lived false alarm (25.5%). Telemetry shows dominant wavelengths of 9.2, 11.5 and 14.7 px in the far, middle and near bands.

TABLE IV
ABLATION ON THE SYNTHETIC BENCHMARK AT $k = 5$ (FA: FALSE ALARMS; SS: STEADY STATE)

Run	Recall@5	Precision@5	FA/frame	SS recall@5	SS FA/frame	Frames/s
Full system	**92.4%**	**51.7%**	**1.77**	**99.3%**	**1.90**	**2.67**
Without temporal stage	89.2%	40.2%	2.73	88.7%	2.80	2.64
Without glint suppression	92.4%	51.2%	1.81	99.3%	1.94	2.75
Without morphology and shape filter	92.4%	45.7%	2.50	99.3%	2.68	2.65

The temporal stage gives the largest gain: false alarms per frame fall by 35% (2.73 to 1.77) and precision rises from 40.2% to 51.7%. Morphology and the shape filter lower false alarms from 2.50 to 1.77 by rejecting foam and debris. Glint suppression has little effect here (1.81 to 1.77) because the synthetic glitter specks are 1–3 px and are already removed by opening and the minimum-area limit; larger glint patches in real footage are expected to make it matter more.

C. Shortlist Length

Persistence lifts people above short-lived clutter, so recall@3 rises from 72.4% to 92.3% (Fig. 7). The cost is the warm-up: nothing is shown in the first 4 frames, so the full system plateaus at 92.4% and the run without tracking overtakes it for $k \geq 7$. Most frames contain 2–3 people, so recall@1 cannot exceed about 45%.

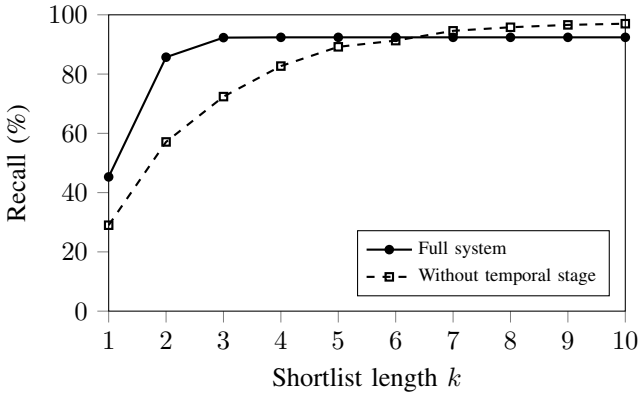


Fig. 7. Recall against shortlist length (360 frames, 739 people).

Fig. 8 shows why tracking works: sequence 5 produced 665 raw detections, yet each person stayed on the shortlist for 56 frames and no other track lasted more than 7. Recall@5 was stable across altitude (93.2% below 60 m, 92.9% at 60–100 m, 89.0% at 100–140 m, 93.3% at 140 m and above) and target size (92.2% under 16 px, $n = 618$; 93.4% for 16–32 px,

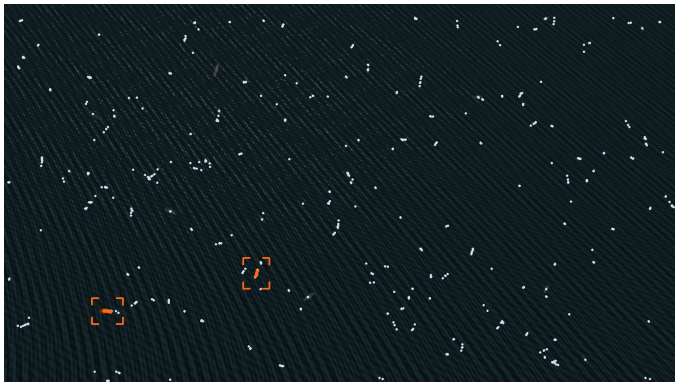


Fig. 8. All detections across 60 frames of sequence 5. The two people form continuous tracks (orange); other detections appear briefly (pale dots).

$n = 121$). Of 11 false alarms inspected at frame 31 across the six sequences, 3 resembled debris, 3 whitecaps and 5 showed nothing visible.

VIII. Limitations

- Results are synthetic only; loaders are ready for SeaDronesSee [8], MOBDrone [9] and AFO [10].
- Dark, colourless targets give weak colour and texture contrast, and very bright white clothing can be removed as glint.
- Coastline or a large ship in a reference window can contaminate the reference.
- Long-lived whitecaps or debris can be promoted to a track.
- Phase correlation models translation only, not rotation or zoom.
- Parameters are hand-tuned; night and thermal imagery are out of scope.

IX. Conclusion

Modelling the sea rather than the person lets a purely classical pipeline find small survivors without training data. Texture anomaly scoring against a robust local reference finds candidates, and colour, shape and temporal tests remove most clutter, giving 92.4% recall@5 and 99.3% steady-state recall on a synthetic benchmark at 1.77 false alarms per frame. Every stage is visible and tunable in both front ends. The next step is evaluation on the public SeaDronesSee, MOBDrone and AFO datasets, followed by homography-based motion compensation.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Pearson Education, 2012.

[2] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2011.

[3] M. Sonka, V. Hlavac, and R. Boyle, *Digital Image Processing and Computer Vision*, India ed. Cengage Learning, 2009.

[4] J. Howse, P. Joshi, and M. Beyeler, *OpenCV: Computer Vision Projects with Python*. Packt Publishing, 2016.

[5] D. L. Baggio, S. Emami, D. M. Escrivá, K. Ievgen, N. Mahmood, J. Saragih, and R. Shilkrot, *Mastering OpenCV 3*, 2nd ed. Packt Publishing, 2017.

[6] R. Laganière, *OpenCV Computer Vision Application Programming Cookbook*, 2nd ed. Packt Publishing, 2014.

[7] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, Jul. 2002, doi: 10.1109/TPAMI.2002.1017623.

[8] L. A. Varga, B. Kiefer, M. Messmer, and A. Zell, "SeaDronesSee: A maritime benchmark for detecting humans in open water," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2022, pp. 2260–2270.

[9] D. Cafarelli *et al.*, "MOBDrone: A drone video dataset for man overboard rescue," in *Image Analysis and Processing – ICIAP 2022*. Cham, Switzerland: Springer, 2022, pp. 633–644.

[10] J. Gąsienica-Józkowy, M. Knapik, and B. Cyganek, "An ensemble deep learning method with optimized weights for drone-based water rescue and surveillance," *Integr. Comput.-Aided Eng.*, vol. 28, no. 3, pp. 221–235, 2021, doi: 10.3233/ICA-210649.